المتطلب الثاني: التكامل بين Laravel وخدمة المخزون عبر gRPC

يوثّق هذا الملف البنية التي يعتمدها المشروع لتنفيذ التواصل بين خدمتين مستقلتين، ومسار قراءة المخزون وحجزه وإرجاعه، وأماكن الكود والاختبارات.

1. الفكرة المعمارية

يتكوّن الحل من خدمتين لكل منهما مسؤولية وقاعدة بيانات مستقلة:

```
المتصفح
   |
   v
واجهة React  --->  Laravel API
                       | \
                       |  \-- MySQL الخاصة بـ Laravel
                       |      المنتجات، المستخدمون، السلة، الطلبات
                       |
                       | gRPC / Protocol Buffers
                       v
                  Java Spring Boot
                       |
                       \-- MySQL الخاصة بالمخزون
                           الكميات والحجوزات
```

- تملك Laravel بيانات الكتالوج والمستخدمين والطلبات.
- تملك خدمة Java الكمية المتاحة وسجل الحجوزات، وهي المصدر المعتمد للمخزون عند تفعيل `ORDER_STOCK_MANAGER=grpc`.
- تتشارك الخدمتان `product_id` بوصفه معرّفًا مرجعيًا؛ خدمة المخزون لا تقرأ قاعدة Laravel ولا تعتمد على اتصال بها.
- يعرّف ملف Protocol Buffers الرسائل والعمليات المشتركة. تتواصل Laravel مع Java عبر عميل gRPC، وليس عبر استعلامات SQL مشتركة.

2. مواقع الملفات الأساسية

```
docker-compose.yml
├── mysql-primary                قاعدة Laravel للكتابة
├── mysql-replica                نسخة القراءة لقاعدة Laravel
├── inventory-db                 قاعدة المخزون المستقلة
├── backend                      Laravel API وعميل gRPC
└── inventory-service            خدمة Java/gRPC

app/
├── Contracts/Inventory/
│   └── InventoryGateway.php     عقد بوابة المخزون في Laravel
└── Services/Inventory/
    └── GrpcInventoryGateway.php  استدعاءات gRPC ومهلة الاتصال

Modules/
├── Product/app/Services/
│   └── ProductCatalogService.php قراءة الكتالوج وتهيئة/قراءة المخزون
└── Order/
    ├── app/Services/
    │   ├── GrpcStockManager.php  حجز المنتجات وتحرير الحجز الجزئي
    │   └── OrderService.php      حفظ الطلب ثم تثبيت الحجز أو تعويضه
    └── tests/
        ├── Feature/              اختبارات الوحدة والتكامل الوهمي
        └── Integration/          اختبارات حية لخدمة gRPC

inventory-service/src/main/
├── proto/inventory.proto         عقد gRPC
├── java/.../grpc/
│   └── InventoryGrpcService.java تنفيذ نقاط gRPC
├── java/.../service/
│   └── InventoryBusinessService.java منطق المخزون والمعاملات
└── resources/schema.sql          جداول قاعدة المخزون

tests/
├── bootstrap.php                 حاجز أمان لقاعدة الاختبار
└── Feature/TestingDatabaseIsolationTest.php
```

3. عقد gRPC

العقد موجود في `inventory-service/src/main/proto/inventory.proto`. العمليات الأساسية هي:

العملية	الغرض
`GetStockQuantity`	قراءة الكمية المتاحة لمنتج
`EnsureProductStock`	إنشاء سجل مخزون ابتدائي إن لم يكن موجودًا
`ReserveStock`	حجز كمية وربطها بمعرّف حجز
`CommitReservation`	تثبيت الحجز بعد نجاح حفظ الطلب
`ReleaseReservation`	إلغاء الحجز وإعادة كمياته المتاحة
`RollbackStock`	تعويض كمية محددة عند الحاجة

مقتطف من العقد:

```
service InventoryService {
  rpc GetStockQuantity (StockRequest) returns (StockResponse);
  rpc EnsureProductStock (ReserveRequest) returns (ReserveResponse);
  rpc ReserveStock (ReserveRequest) returns (ReserveResponse);
  rpc CommitReservation (ReserveRequest) returns (ReserveResponse);
  rpc ReleaseReservation (ReserveRequest) returns (ReserveResponse);
  rpc RollbackStock (RollbackRequest) returns (RollbackResponse);
}
```

يحمل `ReserveRequest.order_ref_id` معرّف الحجز. ينشئ Laravel هذا المعرّف لكل طلب، ويستخدمه في الحجز والتثبيت أو التحرير، ما يجعل دورة الحجز قابلة للتتبع في قاعدة المخزون.

4. فصل قواعد البيانات

يعرّف `docker-compose.yml` خدمتي MySQL وvolume مستقلًا لكل واحدة:

- `mysql-primary` تستخدمه Laravel لكتابات بيانات التطبيق، و `mysql-replica` لقراءات التطبيق.
- `inventory-db` تستخدمه Java للمخزون والحجوزات.
- يتصل `backend` بعنوان gRPC الداخلي `inventory-service:9090` .
- تتصل خدمة Java بـ `inventory-db:3306` فقط.
- volume قاعدة المخزون هو `inventory_mysql_data` ، وقاعدتا Laravel تستخدمان `mysql_primary_data` و `mysql_replica_data` .

تحتوي `schema.sql` على:

- `inventory_items` : الكمية المتاحة لكل `product_id` .
- `reservations` : حالة الحجز (`RESERVED` أو `COMMITTED` أو `RELEASED`).
- `reservation_items` : المنتجات والكميات المرتبطة بكل حجز.

عند إنشاء منتج أو عرضه، تستدعي Laravel `EnsureProductStock` . تنشئ هذه العملية سجلًا ابتدائيًا عند غيابه ولا تستبدل كمية سجل المخزون الموجود؛ لذلك يبقى المخزون المخفّض أو المحجوز ملكًا لخدمة Java.

5. قراءة المخزون من الكتالوج

تقرأ `ProductCatalogService` بيانات المنتج وسعره من قاعدة Laravel، ثم تستعلم عن الكمية من خدمة المخزون عندما يكون مدير المخزون مضبوطًا على gRPC:

```
$this->inventory->ensureProductStock($productId, (int) $product->stock);
$snapshot['stock'] = $this->inventory->getAvailableQuantity($productId);
```

قيمة `products.stock` في Laravel هي كمية ابتدائية لتهيئة سجل غير موجود. بعد ذلك تكون كمية المخزون المعروضة هي الكمية التي تعيدها خدمة المخزون.

6. دورة إنشاء الطلب

ينفذ `OrderService::createOrder` الخطوات التالية:

1. ينشئ معرّف حجز فريدًا.
2. يحجز جميع كميات الطلب عبر `GrpcStockManager` .
3. يحفظ الطلب وبنوده داخل transaction في قاعدة Laravel.
4. إذا فشل الحفظ، يطلب تحرير الحجز من خدمة المخزون؛ وإذا فشل التحرير أيضًا، يسجل خطأً حرجًا ظاهرًا.
5. إذا نجح الحفظ، يثبت الحجز عبر `CommitReservation` .

مقتطف مبسط من منطق التعويض في Laravel:

```
try {
    $order = DB::transaction(function () use ($userId, $items) {
        // قراءة بيانات المنتجات وحفظ الطلب وبنوده في قاعدة Laravel.
    });
} catch (\Throwable $exception) {
    $this->stockManager->release($reservationId);
    throw $exception;
}

$this->stockManager->commit($reservationId);
```

هذه معاملة موزعة بتعويض (Saga): قاعدة Laravel وقاعدة المخزون لا تدخلان في transaction مشتركة. لذلك يحجز النظام المخزون أولًا، ثم يحفظ الطلب، ثم يثبت الحجز؛ ويحرر الحجز إذا فشل حفظ الطلب.

7. التزامن والمهلات والأخطاء

- يقفل Java سجل كمية المنتج داخل transaction قبل فحص المتاح وتخفيضه. يمنع هذا تجاوز الكمية عند وصول حجوزات متزامنة على آخر قطعة.
- معاملات قاعدة المخزون تحفظ الحجز وبنوده في `inventory-db` بدل تخزينها في ذاكرة Laravel.
- لكل استدعاء gRPC مهلة **5 ثوانٍ** في `GrpcInventoryGateway` .
- أخطاء gRPC ترجع إلى التطبيق كأخطاء صريحة؛ لا تُحوّل إلى نجاح وهمي.
- يحرر `GrpcStockManager` الحجز إذا فشل حجز أحد المنتجات بعد نجاح حجز منتجات أخرى.
- يحتفظ الحجز بحالة تمنع تحرير الحجز المثبّت أو خصم الكمية نفسها مرتين.

8. الاختبارات

الاختبارات العادية

```
composer test
```

يضمن `tests/bootstrap.php` تشغيل الاختبارات على SQLite داخل الذاكرة، مع `ORDER_STOCK_MANAGER=local`. يوقف PHPUnit التشغيل قبل إقلاع Laravel إذا لم تتوفر SQLite أو كانت متغيرات قاعدة الاختبار غير آمنة. لا تشغّل الاختبارات على قاعدة MySQL الحية.

اختبارات gRPC الحية

شغّل خدمات Docker أولًا:

```
docker compose up -d
docker compose ps
```

ثم نفّذ:

```
docker compose run --rm --no-deps `
  -e RUN_INVENTORY_INTEGRATION_TESTS=1 `
  backend php artisan test --testsuite=Integration
```

تختبر المجموعة الحية:

- قراءة المخزون وحجزه وتحريره عبر gRPC.
- طلبين متوازيين على آخر قطعة؛ تكرر الاختبار السباق خمس مرات، ويتوقع فائزًا واحدًا في كل مرة.
- إعادة المخزون إذا فشل حفظ الطلب في Laravel.
- فشل الاتصال بخدمة غير متاحة.
- انتهاء مهلة فعلية لخدمة لا تستجيب، ضمن حد الخمس ثواني.
- تثبيت الحجز ومنع تحريره بعد تثبيته.

تستخدم اختبارات التكامل معرّفات منتجات اصطناعية عشوائية ولا تنشئ منتجات في كتالوج Laravel. تبقى سجلات الاختبار الاصطناعية في قاعدة المخزون حتى لا تعبث الاختبارات ببيانات التطبيق.

9. الإعداد والتشغيل

في Docker يضبط Compose تلقائيًا:

```
ORDER_STOCK_MANAGER=grpc
GRPC_INVENTORY_HOST=inventory-service:9090
```

أما التشغيل المحلي، فيحتاج أن تكون قاعدة خدمة المخزون مستقلة وأن تعمل خدمة Java على المنفذ `9090` ، ثم تضبط Laravel:

```
ORDER_STOCK_MANAGER=grpc
GRPC_INVENTORY_HOST=127.0.0.1:9090
```

بعدها يمكن تشغيل Laravel وواجهة React بالطريقة المعتادة للمشروع.

10. دليل التنفيذ والاختبارات

- عقد الاتصال: `inventory-service/src/main/proto/inventory.proto` .
- إعداد قواعد البيانات والخدمات: `docker-compose.yml` .
- إعداد الجداول: `inventory-service/src/main/resources/schema.sql` .
- بوابة Laravel إلى gRPC: `app/Services/Inventory/GrpcInventoryGateway.php` .
- منطق الحجز في Laravel: `Modules/Order/app/Services/GrpcStockManager.php` .
- تدفق إنشاء الطلب والتعويض: `Modules/Order/app/Services/OrderService.php` .
- التنفيذ والمعاملات في Java: `inventory-service/src/main/java/com/example/inventory_service/service/InventoryBusinessService.java` .
- اختبارات العميل المعزولة: `Modules/Order/tests/Feature/InventoryGrpcTest.php` .
- اختبارات الخدمة الحية: `Modules/Order/tests/Integration/InventoryGrpcIntegrationTest.php` .

---

11. شرح المتطلب كما ورد في العرض

المتطلب الثاني هو **التواصل بين الخدمات** (Inter-Service Communication). الفكرة المطلوبة ليست إضافة شاشة مخزون فقط، بل فصل إدارة المخزون في خدمة مستقلة مكتوبة بلغة/إطار مختلف، لها قاعدة بياناتها، ويتواصل معها التطبيق الأساسي عبر عقد gRPC مشترك.

التدفقان اللذان يجب أن تعرف شرحهما:

1. **عرض تفاصيل منتج:** يسأل التطبيق خدمة المخزون عن الكمية المتاحة ثم يضمها إلى تفاصيل المنتج.
2. **إنشاء طلب:** يرسل التطبيق حجز كميات منتجات الطلب. إذا نجح الحجز يكمل حفظ الطلب، وإذا فشل لا ينشئه. وإذا فشل حفظ الطلب بعد الحجز، يعيد الكمية إلى المخزون.

ويطلب العرض إظهار حالتي فشل مهمتين: خدمة مخزون متوقفة/بطيئة مع مهلة واضحة، وطلبان متزامنان على آخر قطعة بحيث ينجح واحد فقط. في العرض اختُصرت العمليات إلى **معرفة الكمية، حجز الكمية، وإرجاعها**. أما التطبيق الحالي فيضيف عمليات تهيئة سجل المخزون، تثبيت الحجز، وتحرير الحجز بالمعرّف؛ هذه إضافات لتقوية دورة الحجز وليست اختلافاً عن الهدف الأساسي.

12. ما هو gRPC بالضبط؟

gRPC إطار اتصال لاستدعاء دوال خدمة بعيدة (Remote Procedure Call). بدلاً من أن يبني العميل طلب HTTP يدوياً ويحلل JSON، يستدعي دالة يعرّفها العقد وكأنها دالة محلية:

```
Laravel: ReserveStock(request) ── gRPC/HTTP/2 ──> Java: reserveStock(request)
```

لكنها ليست دالة محلية فعلياً: تمر البيانات عبر الشبكة، وقد تفشل الخدمة أو الشبكة أو تنتهي المهلة. لذلك يجب التعامل مع الاتصال باعتباره عملية موزعة لها أخطاء وزمن انتظار وحالة نجاح مستقلة.

العناصر الأساسية

المفهوم	معناه في مشروعنا
Protocol Buffers / Protobuf	لغة تعريف الرسائل والخدمات في `inventory.proto`، وصيغة ثنائية فعالة لتسلسل الرسائل.
Service / RPC	العمليات التي تعرضها خدمة المخزون مثل `GetStockQuantity` و `ReserveStock`.
Message	شكل الطلب أو الرد، مثل `ReserveRequest` و `ReserveResponse`.
Stub / Client	كود مولّد يوفر للعميل دوال استدعاء الخدمة؛ Laravel يستخدم `InventoryServiceClient`، وJava تستخدم `InventoryServiceGrpc`.
HTTP/2	النقل الذي يستخدمه gRPC، مع framing ثنائي وميزات مثل تعدد الاستدعاءات على الاتصال.
Deadline / Timeout	الحد الأقصى الذي ينتظره العميل لرد RPC؛ عند تجاوزه يجب إيقاف الانتظار والتعامل مع الفشل.
gRPC status	حالة النقل مثل `NOT_FOUND` أو `INVALID_ARGUMENT` أو `INTERNAL` أو `DEADLINE_EXCEEDED`.
Unary RPC	استدعاء واحد يرسل طلباً واحداً ويتلقى رداً واحداً. جميع عملياتنا الحالية من هذا النوع، وليست streaming.

مقارنة سريعة مع REST في المشروع

REST مناسب لواجهات التطبيق التي يستهلكها المتصفح أو تطبيقات عامة؛ مشروعنا يستخدم Laravel API لهذا الغرض. أما اتصال Laravel الداخلي بخدمة المخزون فهو gRPC بعقد Protobuf وعميل مولّد. كلاهما يمر عبر الشبكة، لكن gRPC هنا يحقق عقداً محدداً بين خدمتين بلغتين مختلفتين. لا يتصل المتصفح بخدمة Java مباشرة؛ الواجهة الأمامية تتواصل مع Laravel.

13. البنية: من يملك ماذا؟

```
React / المتصفح
      | REST/JSON
      ▼
Laravel API ── قاعدة Laravel (المستخدمون، الطلبات، المنتجات)
      |
      | gRPC: HTTP/2 + Protobuf، العنوان inventory-service:9090
      ▼
Java Spring Boot ── قاعدة inventory-db (الكميات والحجوزات)
```

- هذه **خدمة متعددة اللغات** (Polyglot Service): Laravel/PHP هو التطبيق الأساسي، وخدمة المخزون Java/Spring Boot.
- لكل خدمة مسؤولية وملكية بيانات مستقلة. قاعدة Laravel لا تُستعلم من Java، وJava لا تتصل بقاعدة Laravel.
- `product_id` هو معرّف مرجعي مشترك فقط؛ ليس مفتاحاً خارجياً بين قاعدتين ولا يعني مشاركة قاعدة البيانات.
- خدمة Java هي **مصدر الحقيقة للمخزون** عند `ORDER_STOCK_MANAGER=grpc`. قيمة `products.stock` في Laravel تستخدم كقيمة ابتدائية لإنشاء سجل مخزون مفقود فقط.
- `InventoryGateway` يفصل منطق Laravel عن وسيلة الاتصال. يربط `OrderServiceProvider` العقد إما بـ `LocalStockManager` أو `GrpcStockManager` وفق الإعداد؛ بذلك تبقى واجهة الطلبات غير معتمدة مباشرة على تفاصيل gRPC.

14. العقد المشترك والرسائل

هذا مقتطف فعلي من `inventory-service/src/main/proto/inventory.proto`:

```proto
syntax = "proto3";
package inventory;

service InventoryService {
  rpc GetStockQuantity (StockRequest) returns (StockResponse);
  rpc EnsureProductStock (ReserveRequest) returns (ReserveResponse);
  rpc ReserveStock (ReserveRequest) returns (ReserveResponse);
  rpc CommitReservation (ReserveRequest) returns (ReserveResponse);
  rpc ReleaseReservation (ReserveRequest) returns (ReserveResponse);
  rpc RollbackStock (RollbackRequest) returns (RollbackResponse);
}

message StockRequest {
  int64 product_id = 1;
}

message ReserveRequest {
  int64 product_id = 1;
  int32 quantity = 2;
  string order_ref_id = 3;
}
```

- أرقام الحقول (`= 1`، `= 2`، ...) هي **أرقام Protobuf على السلك** وليست قيماً افتراضية. يجب عدم إعادة استخدام رقم حقل حُذف في إصدار لاحق؛ تغييره بلا توافق قد يكسر العملاء القدامى.

- `int64` مناسب لمعرف المنتج في Java `long`، و `int32` للكمية في Java `int`، و `string order_ref_id` يحمل معرّف الحجز.
- كل عملية هنا RPC أحادية الطلب/الرد. رغم أن المشروع يشغّل خدمة تبدو لحظية، فهذا لا يجعل كل استدعاء streaming أو غير متزامن.
- عميل PHP والـJava stubs مولّدان من العقد؛ ملف العقد هو المرجع المشترك، لا ينبغي تعديل ملفات الكود المولّد يدوياً.
- `RollbackStock` موجود في العقد، لكن `InventoryGateway` ومسار الطلب الحالي يستخدمان `ReleaseReservation` لتعويض الحجز. لا تقل في المقابلة إن كل RPC معلن في العقد مستدعًى من تدفق Laravel.

15. كيف يصل الاستدعاء إلى المخزون؟

خادم Java

`InventoryGrpcService` هو طبقة النقل: يقرأ حقول الطلب، يستدعي `InventoryBusinessService`، يبني الرد أو يحوّل الخطأ إلى gRPC status. مثال مبسّط من التنفيذ:

```
@Override
public void getStockQuantity(
        StockRequest request,
        StreamObserver<StockResponse> responseObserver) {
    long productId = request.getProductId();
    int quantity = inventoryBusinessService.getAvailableQuantity(productId);

    responseObserver.onNext(StockResponse.newBuilder()
            .setProductId(productId)
            .setAvailableQuantity(quantity)
            .build());
    responseObserver.onCompleted();
}
```

`StreamObserver` هو واجهة إرسال الرد: `onNext` يرسل قيمة الرد، و `onCompleted` ينهي RPC. وللأخطاء يستخدم التنفيذ `responseObserver.onError(Status...asRuntimeException())`. من أمثلة التحويل في المشروع: منتج غير موجود إلى `NOT_FOUND`، ومدخل غير صالح إلى `INVALID_ARGUMENT`، وخطأ داخلي إلى `INTERNAL`.

عميل Laravel

`GrpcInventoryGateway` هو المحوّل بين عقد Laravel وعميل gRPC المولّد:

```
$client = new InventoryServiceClient($host, [
    'credentials' => ChannelCredentials::createInsecure(),
]);

[$response, $status] = $client->{$method}(
    $request,
    [],
    ['timeout' => 5_000_000]
)->wait();

if ($status->code !== 0) {
    throw new RuntimeException(
        "Inventory gRPC request failed ({$status->code}): "
        . ($status->details ?: 'Unknown error')
    );
}
```

المهلة هنا بوحدة **الميكروثانية**: `5_000_000` = خمس ثوانٍ. استخدام `wait()` يعني أن مسار Laravel ينتظر نتيجة هذا الاستدعاء قبل إكمال العملية؛ لا يعني أنه يطلق عملاً خلفياً. يفحص المحوّل أيضاً وجود امتداد PHP `grpc` ، ويتحقق من نوع الرد، ولا يحوّل تعطل الشبكة إلى نجاح وهمي.

16. تدفق قراءة تفاصيل المنتج

`ProductCatalogService::snapshotWithInventory` يقرأ بيانات المنتج من قاعدة Laravel، ثم عند تفعيل gRPC:

```
$this->inventory->ensureProductStock($productId, (int) $product->stock);
$snapshot['stock'] = $this->inventory->getAvailableQuantity($productId);
```

الترتيب مهم:

1. `EnsureProductStock` ينشئ صف المخزون **إذا لم يوجد فقط** (`INSERT IGNORE` في Java).
2. بعدها `GetStockQuantity` يقرأ الرصيد المملوك لخدمة المخزون.
3. قيمة `products.stock` لا تستبدل رصيداً موجوداً، وإلا لأعاد عرض المنتج المخزون إلى قيمته القديمة كل مرة.

ميزة التصميم: المنتج ومخزونه لهما مصدران واضحان. تكلفة التصميم: إظهار المنتجات قد يتضمن RPC إضافياً لكل منتج في التنفيذ الحالي؛ لذلك عند حجم كبير نحتاج التفكير في batch RPC/واجهة تجميع أو cache مع سياسة freshness واضحة، دون اعتبار بيانات cache بديلاً عن فحص الحجز النهائي.

17. تدفق إنشاء الطلب: من السلة إلى الحجز

في `OrderService::createOrder` :

1. ينشأ UUID واحد بوصفه `reservationId` .
2. يستدعى `StockManager::reserve` **قبل** إنشاء الطلب.
3. `GrpcStockManager` يتحقق أن المعرّفات والكميات موجبة، ويجمع الأسطر المتكررة لنفس المنتج.
4. يحجز كل منتج عبر `ReserveStock` مستخدماً `reservationId` نفسه.

5. إذا رُفض أي منتج، لا يكمل `OrderService` إلى إنشاء الطلب.
6. بعد نجاح كل الحجوزات، تحفظ Laravel الطلب وبنوده داخل `DB::transaction`.
7. إذا نجح حفظ الطلب، تستدعى `CommitReservation`.

شبه كود مطابق للفكرة:

```
$reservationId = (string) Str::uuid();
$reservation = $this->stockManager->reserve($reservationId, $items);

if (!$reservation['success']) {
    throw new RuntimeException($reservation['message']);
}

try {
    $order = DB::transaction(function () use ($userId, $items) {
        // قراءة المنتجات وحساب الأسعار وحفظ الطلب وبنوده.
    });
} catch (Throwable $exception) {
    $this->stockManager->release($reservationId);
    throw $exception;
}

$this->stockManager->commit($reservationId);
```

هذا ليس transaction واحدة تشمل قاعدتي البيانات؛ `DB::transaction` يحمي تغييرات قاعدة Laravel فقط، و`@Transactional` في Java يحمي تغييرات قاعدة المخزون فقط.

18. لماذا الحجز ثم Commit/Release؟ (Saga)

لا توجد معاملة ذرية موزعة تربط MySQL الخاصة بـLaravel وMySQL الخاصة بالمخزون في هذا المشروع. لذلك نستخدم نمط Saga / **تعويض** (Compensation):

```
Reserve (Java) → Save order (Laravel) → Commit reservation (Java)
                          |
                          └── عند الفشل: Release reservation (Java)
```

- Reserve: يخفض المتاح ويسجل الحجز في قاعدة المخزون.
- **نجاح حفظ الطلب:** تثبيت حالة الحجز `COMMITTED`.
- **فشل حفظ الطلب:** تحرير الحجز وإعادة كمياته.
- لا نُرجع نجاحاً للمستخدم إذا رُفض المخزون أو فشل RPC.

نقطة مقابلة مهمة: إذا نجح حفظ الطلب ثم فشل اتصال `CommitReservation`، فالطلب يبقى محفوظاً لكن Laravel يسجل `critical` ويرمي خطأ يفيد بأن التأكيد تعذر. هذه نافذة فشل بين خدمتين، ولا يحلها transaction المحلية. الحل الإنتاجي الأكثر متانة عادةً يشمل retry آمن/جدولة مصالحة أو Transactional Outbox ومراقبة الحجوزات العالقة. هذا التطوير غير منفذ هنا، لذا اذكره كحدّ حالي لا كميزة موجودة.

19. التزامن ومنع بيع آخر قطعة مرتين

قراءة الرصيد ثم خصمه في خطوتين غير محميتين تسبب Race Condition:

```
طلب A يقرأ 1 ─┐
              ├─ كلاهما يعتقد أن القطعة متاحة، وقد يصبح الرصيد سالباً
طلب B يقرأ 1 ─┘
```

في `InventoryBusinessService.reserveStock` تنفذ Java العملية داخل `@Transactional`:

```
availableQuantity = jdbcTemplate.queryForObject(
        "SELECT available_quantity FROM inventory_items "
                + "WHERE product_id = ? FOR UPDATE",
        Integer.class,
        productId
);

if (availableQuantity < quantity) {
    return ReservationResult.failure("Insufficient stock.");
}

jdbcTemplate.update(
        "UPDATE inventory_items SET available_quantity = available_quantity - ? "
                + "WHERE product_id = ?",
        quantity,
        productId
);
```

`SELECT ... FOR UPDATE` يقفل صف المنتج حتى نهاية transaction. إذا طلبان حاولا حجز آخر قطعة، ينتظر أحدهما قفل الآخر؛ بعد أن يخفض الأول الرصيد ويعتمد transaction، يرى الثاني الرصيد المحدّث ويرفض. يوجد أيضاً قفل صف الحجز `FOR UPDATE` وجداول بقيود مفاتيح أساسية/أجنبية؛ وقيد `available_quantity >= 0` دفاع إضافي على مستوى قاعدة البيانات.

الاختبار الحي `test_only_one_live_grpc_reservation_wins_a_race_for_the_last_item` يطلق عاملين متوازيين، ويكرر السيناريو خمس مرات، ويتحقق من فائز واحد فقط ورصيد صفر.

20. معرّف الحجز، الحالات، وIdempotency

معرّف الحجز UUID هو مفتاح الربط بين طلب Laravel وعملية المخزون، ويُستخدم في reserve/commit/release. في قاعدة خدمة المخزون:

```
reservations:       id, status
reservation_items:  reservation_id, product_id, quantity, released
inventory_items:    product_id, available_quantity
```

الحالات هي:

```
RESERVED ──commit──> COMMITTED
    │
    └──release──> RELEASED
```

- إعادة `commit` لحجز `COMMITTED` تعيد نجاحاً؛ وإعادة `release` لحجز `RELEASED` لا تعيد الكمية مرة ثانية.
- لا يمكن تحرير حجز `COMMITTED` .
- إذا أُعيد طلب حجز نفس المنتج بنفس الكمية ضمن الحجز نفسه وهو نشط، يتعامل التنفيذ معه كإعادة آمنة؛ أما اختلاف الكمية فيُرفض.
- هذا يجعل بعض العمليات idempotent: تكرارها لا يكرر الأثر الجانبي. لا يعني ذلك أن كل خطأ شبكة يعني أن العملية لم تنفذ؛ قد تنفذ Java ثم ينقطع الرد، لذلك أهمية المعرّف الثابت وإعادة المحاولة الآمنة.
- `reservation_items.released` يمنع إعادة الكمية أكثر من مرة، كما يسمح لـ `RollbackStock` بتحرير بند محدد لو استُخدم.

21. الأخطاء والمهلات وحدود الفشل

فرّق في المقابلة بين نوعين:

1. **رفض وظيفي**: الخدمة ردت بشكل صحيح لكن المخزون غير كافٍ أو المنتج غير موجود. مثال: `ReserveResponse.success = false` أو gRPC `NOT_FOUND` .
2. **فشل نقل/تشغيل**: الخدمة غير متاحة، الشبكة معطلة، الرد غير صالح، أو انتهت المهلة. يكتشفها عميل Laravel كخطأ gRPC ويرفع `RuntimeException` .

السلوك المطلوب:

- المخزون غير الكافي: إيقاف إنشاء الطلب وإرجاع سبب واضح.
- فشل حجز منتج بعد حجز منتجات أخرى: تحرير الحجز الجزئي كاملاً.
- فشل حفظ الطلب بعد الحجز: طلب تحرير المخزون، ثم إعادة خطأ الحفظ.
- فشل التحرير التعويضي: تسجيل `critical` وإظهار أن هناك حجزاً يحتاج معالجة؛ لا نخفي الفشل.
- خدمة بطيئة أو متوقفة: deadline بخمس ثوانٍ بدلاً من الانتظار بلا نهاية. المهلة لا تلغي بالضرورة عملية سبق أن نفذتها الخدمة؛ يلزم التعامل مع إعادة المحاولة باستخدام معرّف حجز ثابت.
- `ChannelCredentials::createInsecure()` يعني أن الاتصال الحالي بلا TLS. مناسب لتطوير Docker المحلي على شبكة المشروع، لكنه ليس إعداد إنتاج آمن عبر شبكة غير موثوقة؛ الإنتاج يحتاج TLS (وقد يحتاج mTLS وهوية/تفويض للخدمات).

22. كيف تُبنى وتشغّل القطع معاً؟

- يعرّف Maven dependencies الخاصة بـgRPC وProtobuf، ويولّد Java messages/stubs من `src/main/proto/inventory.proto` أثناء build.

- Composer يتضمن `grpc/grpc` و `google/protobuf` ، و `composer.json` يحمّل كود عميل PHP المولّد عبر PSR-4.
- `inventory-service` يستمع داخل Compose على `9090` ، وLaravel يتصل بـ `inventory-service:9090` عبر اسم الخدمة في شبكة Docker، لا بـ `localhost` .
- `inventory-service` يتصل بـ `inventory-db:3306` فقط. `backend` يضبط `ORDER_STOCK_MANAGER=grpc` ، بينما الافتراضي في الإعداد `local` .
- التحقق من إعداد الخدمات:

```
docker compose up -d
docker compose ps
```

اختبار عميل Laravel مع mocks (لا يحتاج خادم gRPC حي):

```
php artisan test Modules/Order/tests/Feature/InventoryGrpcTest.php
```

اختبارات gRPC الحية تحتاج امتداد PHP `grpc` وتشغيل الخدمات:

```
docker compose run --rm --no-deps `
  -e RUN_INVENTORY_INTEGRATION_TESTS=1 `
  backend php artisan test --testsuite=Integration
```

تغطي اختبارات التكامل القراءة والحجز والتحرير، سباق آخر قطعة، تعويض فشل حفظ الطلب، توقف الخدمة، deadline لخادم لا يرد، ومنع تحرير الحجز بعد تثبيته. الاختبارات الحية مصممة لتكون اختيارية؛ المتغير `RUN_INVENTORY_INTEGRATION_TESTS=1` هو opt-in.

آلية الاختبار وما دور كل ملف؟

لدينا مستويان مختلفان من الاختبار، ومن المهم عدم الخلط بينهما:

1. **اختبارات** Feature **مع** Mock**:** الملف `Modules/Order/tests/Feature/InventoryGrpcTest.php` يستبدل `InventoryGateway` بكائن Mockery. نحدد مسبقاً رد الخدمة المتوقع، ثم نتحقق من سلوك `GrpcStockManager` من دون تشغيل Java أو فتح اتصال gRPC حقيقي. يختبر النجاح والرفض، المدخلات غير الصالحة، تحرير الحجز الجزئي، المهلة الممثلة باستثناء، وتجميع الأسطر المكررة للمنتج نفسه. هذا سريع ويعزل منطق Laravel، لكنه لا يثبت أن الشبكة أو Java أو قاعدة المخزون تعمل.

2. **اختبارات** Integration **حية:** الملف `Modules/Order/tests/Integration/InventoryGrpcIntegrationTest.php` يستخدم `GrpcInventoryGateway` الحقيقي، ويتصل بخدمة Java وقاعدة مخزون Docker. لا يبدأ افتراضياً: يجب ضبط `RUN_INVENTORY_INTEGRATION_TESTS=1` ، كما يتحقق `setUp()` من وجود امتداد PHP `grpc` . يستخدم `RefreshDatabase` لقاعدة Laravel الاختبارية، ويولّد معرفات منتجات رقمية عشوائية بعيدة عن معرفات الكتالوج لتجنب اصطدامها. سجلات المنتجات الاصطناعية في قاعدة المخزون المنفصلة لا تُحذف تلقائياً، وهو سلوك مذكور في الاختبارات والتوثيق.

تفاصيل سيناريوهات التكامل:

السيناريو	ماذا يفعل الاختبار؟	ماذا يثبت؟
القراءة والحجز والتحرير	يهيئ منتجاً بكمية 1، يقرأها، يحجزها، يتأكد أن المتاح صار 0، ثم يحرر الحجز ويتأكد من عودة الكمية إلى 1.	أن الطلبات والردود والآثار على قاعدة Java تعمل عبر gRPC فعلياً.
التنافس على آخر قطعة	يبدأ عمليتي PHP مستقلتين بالتوازي للمنتج نفسه، لكل منهما reservation ID مختلف؛ يكرر ذلك خمس مرات.	أن فائزاً واحداً فقط يحجز، والثاني يتلقى `Insufficient stock` ، ولا يصبح الرصيد سالباً.
تعويض فشل إنشاء الطلب	يحجز فعلياً، ثم يجعل حفظ الطلب يفشل بسبب مستخدم غير موجود، ويتحقق من عدم وجود طلب ومن عودة المخزون.	أن `OrderService` ينفذ Release بعد فشل معاملة Laravel.
خدمة غير متاحة	يوجه العميل إلى `127.0.0.1:1` ويتحقق أن الاستدعاء يفشل ضمن المهلة المحددة.	أن فشل الاتصال يظهر كخطأ ولا يتحول إلى نجاح.
خادم بطيء	يشغل خادم TCP مساعداً لا يرسل رد gRPC، ثم يقيس زمن انتهاء طلب العميل ويتأكد أنه قريب من 5 ثوانٍ.	أن deadline يوقف الانتظار بدلاً من تعليقه بلا نهاية.
حجز مثبت	يحجز ثم ينفذ Commit ويحاول Release.	أن الحجز `COMMITTED` لا يعاد إلى المخزون.

سكربتات الاختبار المساعدة

- `Modules/Order/tests/Integration/Support/reserve-worker.php` ليس مجموعة الاختبارات نفسها؛ إنه عامل صغير يطلقه اختبار التنافس كعملية PHP منفصلة. يأخذ `productId` و `reservationId` وعنوان الخدمة من arguments، يرسل `ReserveStock` بكمية 1 وبمهلة 5 ثوانٍ، ثم يطبع JSON فيه النجاح والرسالة ومعرف الحجز. تشغيل عاملين معاً يجعل الاختبار متزامناً فعلياً، بدلاً من استدعاءين متتابعين داخل العملية نفسها.
- `Modules/Order/tests/Integration/Support/grpc-blackhole.php` يفتح TCP listener على منفذ محلي عشوائي، يطبع رقم المنفذ إلى الاختبار، ثم يقبل الاتصال ولا يرسل استجابة. يستخدمه اختبار المهلة لمحاكاة endpoint بطيء. هذا خادم مساعد لا يطبق بروتوكول خدمة المخزون ولا يستبدل خدمة Java الحقيقية؛ وظيفته اختبار حد الانتظار فقط، ويوقفه الاختبار داخل `finally` .
- لا يوجد سكربت shell منفصل يشغّل كل الاختبارات؛ PHPUnit/Laravel test runner يشغّل الكلاسات، وملفات PHP المساعدة أعلاه تُستدعى من اختبار التكامل للحالتين اللتين تحتاجان عمليات مستقلة.

للتشغيل محلياً من جذر المشروع:

```
# اختبارات العميل المعزولة؛ لا تحتاج خدمة Java حية
php artisan test Modules/Order/tests/Feature/InventoryGrpcTest.php

# اختبارات التكامل الحية، بعد تشغيل خدمات Docker
docker compose up -d
docker compose run --rm --no-deps `
  -e RUN_INVENTORY_INTEGRATION_TESTS=1 `
  backend php artisan test --testsuite=Integration
```

إذا كانت اختبارات التكامل لا تعمل، افحص بالترتيب: أن الحاويات `inventory-service` و `inventory-db` سليمة (`docker compose ps`)، أن backend يستطيع الوصول إلى `inventory-service:9090` داخل شبكة Compose، أن

امتداد PHP `grpc` موجود في backend، وأن خيار opt-in مضبوط. لا تشغّل الاختبارات الحية على قاعدة بيانات إنتاج؛ استخدم بيئة اختبار معزولة.

قراءة سكربتات الاختبار سطراً بسطر منطقياً

`reserve-worker.php` : عامل حجز متزامن

يستدعيه اختبار السباق مرتين كعمليتي PHP مستقلتين. تسلسل عمله:

1. يحمل Composer autoloader كي تتوفر الفئات المولّدة وعميل gRPC.
2. يأخذ من سطر الأوامر: رقم المنتج، معرّف الحجز، وعنوان gRPC.
3. ينشئ `InventoryServiceClient` ويرسل `ReserveRequest` لكمية واحدة، مع `order_ref_id` فريد ومهلة خمس ثوانٍ.
4. ينتظر الرد بـ `wait()` . إذا كان gRPC status فاشلاً أو نوع الرد غير صحيح، يكتب إلى STDERR وينتهي برمز خروج غير صفري.
5. إذا وصل رد صحيح، يطبع JSON يتضمن `success` و `message` و `reservation_id` ؛ ويقرأ الاختبار هذا الناتج ويعدّ الحجوزات الناجحة.

نقطة مهمة: العامل لا ينشئ طلب Laravel ولا يختبر واجهة HTTP؛ إنه يضغط نقطة `ReserveStock` مباشرة بخدمة Java كي يعزل اختبار التزامن على مستوى المخزون. الاختبار يشغّل العاملين قبل انتظار أي منهما، ما يسمح بتداخل محاولتي الحجز.

`grpc-blackhole.php` : محاكاة endpoint **لا يرد**

هذا السكربت ليس خادم gRPC كاملاً. ينشئ TCP listener على `127.0.0.1` بمنفذ يختاره نظام التشغيل (`:0`)، يطبع `PORT:<number>` لكي يلتقطه PHPUnit، ثم يقبل الاتصال ولا يرسل أي بيانات بروتوكول. يظل الاتصال معلقاً حتى تنتهي مهلة العميل. يستخدم اختبار التكامل هذا المنفذ مؤقتاً، ويتأكد أن خطأ العميل هو `DEADLINE_EXCEEDED` وأن الزمن قريب من 5 ثوانٍ، ثم يوقف العملية داخل `finally` حتى لو فشل assertion.

إذن هذا يثبت أن **عميل** Laravel **لا ينتظر إلى الأبد عند** endpoint **صامت**؛ ولا يثبت أن خدمة Java نفسها بطيئة أو أن واجهة المستخدم تعرض رسالة بعينها. لا تشغّل هذا السكربت يدوياً كخدمة للمشروع؛ PHPUnit هو الذي يديره وينهيه.

كيف نقرأ نتيجة الاختبارات؟

- `PASS` في Feature يعني أن منطق Laravel تصرف كما اتفقنا مع الـMock؛ ليس دليلاً على اتصال Java.
- `PASS` في Integration يعني أن اختبار التكامل الحي وصل لخدمة المخزون وحقق assertions لذلك السيناريو.
- `SKIPPED` لا يعني نجاح التكامل؛ يعني أن الاختبار الحي لم يُشغّل لأن متغير `RUN_INVENTORY_INTEGRATION_TESTS` غير مضبوط.
- فشل تهيئة الاختبار بسبب غياب امتداد `grpc` أو توقف الحاويات يعني أن البيئة غير جاهزة، وليس بالضرورة أن منطق الحجز خاطئ.

هل المتطلب الثاني ناقص؟

بمقارنة العرض مع الكود والاختبارات، **الوظائف الجوهرية موجودة**: خدمة Java مستقلة بلغة مختلفة وقاعدة بيانات مستقلة، العقد Protobuf، قراءة المخزون مع تفاصيل المنتج، منع إنشاء الطلب عند رفض الحجز، تعويض فشل حفظ الطلب، المهلة والخطأ عند تعطل الاتصال، واختبار تنافس يثبت فائزاً واحداً على آخر قطعة.

لكن توجد نقاط يجب وصفها بدقة في المقابلة، إحداها قد تعدّ نقصاً إذا فُهم نص العرض حرفياً:

بند العرض	حالة التنفيذ	التقييم
خدمة مخزون منفصلة بلغة/إطار مختلف وقاعدة خاصة، والتواصل عبر gRPC	Java/Spring Boot مع `inventory-db` وLaravel يتصل بعنوان الخدمة عبر gRPC	**مكتمل**
معرفة الكمية وعرضها مع تفاصيل المنتج	`ProductCatalogService` يهيئ السجل عند الحاجة ثم يستدعي `GetStockQuantity`	**مكتمل**
رفض الطلب عند نقص المخزون وإظهار سبب	خدمة Java تعيد رفضاً وظيفياً، وLaravel يوقف حفظ الطلب	**مكتمل وظيفياً**
إعادة الكمية إذا فشل حفظ الطلب بعد الحجز	`OrderService` يستدعي `release` ؛ Java تعيد الكمية وتمنع تكرار التحرير	**مكتمل وظيفياً**
خدمة متوقفة أو بطيئة: مهلة ورسالة خطأ واضحة	deadline خمس ثوانٍ واختبار اتصال غير متاح/endpoint صامت؛ `OrderController` يرجع JSON فشل مع HTTP 409 ورسالة الاستثناء	**السلوك موجود**؛ لكن الاختبار لا يمر عبر HTTP ولا يثبت النص المعروض في React للمستخدم
طلبان متزامنان على آخر قطعة: واحد فقط ينجح	اختبار حي يشغّل عاملَي حجز بالتوازي لخمس جولات ويتحقق من فائز واحد	**منع السباق مثبت على** RPC **الحجز**؛ الاختبار لا ينشئ طلبَي Laravel كاملين عبر API
«طلب واحد لحجز كميات كل منتجات الطلب» بحسب شريحة العقد والتدفق	`ReserveRequest` يحمل منتجاً واحداً، و`GrpcStockManager` يرسل RPC متتابعاً لكل منتج مميز، ثم يحرر الحجز كله إذا فشل أي منتج	**اختلاف محتمل عن النص الحرفي**؛ النتيجة الوظيفية all-or-nothing تتم بالتعويض، لكنها ليست RPC واحدة ذرية لكل عناصر الطلب

ما الذي أنصح بإكماله قبل المقابلة؟

1. **احسموا تفسير عبارة «طلب واحد» مع المدرّس/المقيّم.** إذا المقصود RPC واحد يحمل قائمة عناصر، فالتطبيق الحالي لا يطابق هذا الشكل: يلزم رسالة مثل `ReserveOrderRequest` فيها `reservation_id` و`repeated ReservationItem items`، ورد واحد يقرر نجاح/رفض الحجز كله، وتنفيذ Java يحجز كل العناصر داخل transaction قاعدة المخزون نفسها. هذا تعديل للعقد والطرفين والاختبارات، وليس مجرد إعادة تسمية.
2. **أضيفوا اختبار** HTTP **للمستخدم عند توقف المخزون** إن كان المطلوب إثبات ما يراه المستخدم: استدعاء endpoint إنشاء الطلب، التحقق من status وJSON/رسالة مفهومة، والتأكد من عدم إنشاء طلب. حالياً integration test يغطي استثناء العميل والمهلة، بينما الكنترولر يلتقط الاستثناءات ويرجع 409؛ لا يوجد في هذه الاختبارات assertion مباشر على استجابة endpoint أو واجهة React.

3. **إن أردتم إثبات سيناريو «طلبين» حرفياً، أضيفوا اختباراً متوازياً لإنشاء طلبين عبر `OrderService` أو** API على آخر قطعة، ويتحقق من طلب واحد فقط ومن عدم وجود حجز متروك. الاختبار الحالي قوي لإثبات ذرية الحجز في Java، لكنه لا يغطي كامل مسار HTTP والتحقق والحفظ مرتين.

الخلاصة الدقيقة: لا يظهر نقص في الهدف الأساسي لإدارة المخزون والتواصل أو حالات الفشل الرئيسية. أكبر احتمال لعدم المطابقة هو معنى **«طلب واحد لحجز كل عناصر الطلب»**: تنفيذنا الحالي يرسل عدة RPCs مع تعويض، لا RPC واحدة. ونقص الاختبارات الإضافي هو إثبات استجابة المستخدم وسباق الطلبات على مستوى مسار إنشاء الطلب الكامل. لا تقدّم هاتين النقطتين كأنهما منفذتان؛ اشرح الموجود وحدود الإثبات بوضوح.

23. أسئلة مقابلة وإجابات مختصرة

لماذا اخترتم gRPC؟

لأن الاتصال داخلي ومحدد بين خدمتين، والعقد مطلوب أن يعمل بين PHP وJava. Protobuf يعرّف الرسائل والعمليات مرة واحدة ويولّد العملاء والخوادم، وgRPC يوفر RPC فوق HTTP/2 مع أنواع واضحة ومهلات وحالات نقل. واجهات المتصفح العامة ما زالت عبر Laravel API.

ما الذي يضمن أن Laravel وJava يفهمان الرسائل نفسها؟

ملف `inventory.proto` هو العقد المشترك. كل طرف يولّد منه classes/stubs بلغته. أنواع الحقول وأرقامها جزء من التوافق؛ نضيف حقولاً بأرقام جديدة ولا نعيد استخدام أرقام محذوفة.

لماذا تملك خدمة المخزون قاعدة بيانات منفصلة؟

لعزل الملكية: خدمة المخزون وحدها تعدّل الكمية والحجوزات، ويمكن تغيير تنفيذها دون جعل Laravel يعتمد على جداولها الداخلية. التواصل يكون بالعقد فقط، وليس باستعلام SQL مشترك.

هل `DB::transaction` في Laravel تشمل قاعدة المخزون؟

لا. هي تشمل قاعدة Laravel فقط. Java تدير transaction مستقلة في قاعدة المخزون. ننسق بينهما بسلسلة Saga وتعويض عبر release عند فشل الحفظ.

كيف تمنعون طلبين من شراء آخر قطعة؟

Java تقرأ صف المخزون بـ `SELECT ... FOR UPDATE` داخل transaction، وتتحقق ثم تخصم. الصف المقفول يجعل الطلبات المتنافسة تتسلسل على المنتج؛ الاختبار الحي يثبت فائزاً واحداً.

ماذا لو لم يرد المخزون؟

لكل RPC deadline خمس ثوانٍ. يرفع العميل فشلاً صريحاً ولا ينشئ الطلب من دون حجز مؤكد. إذا كان الاستدعاء ضمن عملية سبق أن حجزت منتجات، يحاول تحرير الحجز الجزئي ويسجل فشل التعويض إن حدث.

لماذا نحتاج `reservationId` بدلاً من `orderId` ؟

الحجز يحصل قبل حفظ الطلب؛ لذلك قد لا يوجد order ID بعد. UUID مستقل موجود منذ أول خطوة ويربط كل عمليات reserve/commit/release، ويساعد على إعادة المحاولة الآمنة والتتبع.

ما الفرق بين `Reserve` , `Commit` , و `Release` ؟

`Reserve` يقلل المتاح ويحفظ أثر الحجز، `Commit` يؤكد أن الطلب حُفظ فلا نسمح بإلغاء الحجز، و `Release` يعيد الكمية عندما يفشل إنشاء الطلب. هذا فصل بين الحجز المؤقت ونجاح الطلب في قاعدة أخرى.

ما الفرق بين `success=false` وgRPC status error؟

الأول رد تطبيقي سليم يحمل رفضاً وظيفياً مثل عدم كفاية المخزون؛ الثاني يعني أن RPC نفسه فشل على مستوى النقل/الخادم مثل `NOT_FOUND` أو `INTERNAL` أو `DEADLINE_EXCEEDED` . على العميل فحص الاثنين.

هل الاتصال الحالي مشفر؟

لا. `createInsecure()` لا يستخدم TLS. يعمل في بيئة التطوير المحلية، لكن نشر الخدمة خارج شبكة موثوقة يتطلب TLS وربما mTLS، إضافة إلى ضبط التفويض وعدم كشف المنفذ للإنترنت.

هل كل استدعاء معلن في `.proto` مستخدم؟

لا. المسارات الحالية تستخدم `GetStockQuantity` , `EnsureProductStock` , `ReserveStock` , `CommitReservation` , و `ReleaseReservation` . `RollbackStock` معلن ومطبق في Java لكنه ليس ضمن واجهة `InventoryGateway` المستخدمة في تدفق الطلب الحالي.

24. ملخص شفهي في دقيقة واحدة

فصلنا إدارة المخزون إلى خدمة Java Spring Boot لها قاعدة MySQL مستقلة، بينما بقيت المنتجات والطلبات في Laravel. العقد `inventory.proto` يعرّف الرسائل وعمليات gRPC، وتُولّد منه stubs للطرفين. Laravel يقرأ تفاصيل المنتج من كتالوجه ثم يحصل على الكمية من المخزون. عند إنشاء طلب ينشئ UUID للحجز، ويحجز كل المنتجات أولاً؛ Java تقفل صف المخزون داخل transaction حتى لا ينجح طلبان على آخر قطعة. بعد نجاح الحجز يحفظ Laravel الطلب محلياً ثم يثبت الحجز. إذا فشل حفظ الطلب يحرر الحجز ويعيد الكمية. لا توجد transaction مشتركة بين القاعدتين، لذلك هذا Saga بتعويض، ولكل استدعاء deadline خمس ثوانٍ مع أخطاء واضحة. اختبرنا كذلك التزامن والتعويض وتعطل الخدمة باختبارات gRPC حية اختيارية.